INSTAGRAM FEATURE:
SUGGESTED FRIENDS

GRAPH (BFS)

When you open the app, Instagram runs BFS on the mutual friends graph to find people exactly two connections away from you.

JAVA

```
List suggestFriends(String user,
        Map> graph) {

    Queue queue = new LinkedList<>();
    Set visited = new HashSet<>();
    List suggestions = new ArrayList<>();
    queue.add(user);
    visited.add(user);

    for (int depth = 0; depth < 2 && !queue.isEmpty(); depth++) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            for (String friend : graph.get(queue.poll())) {
                if (visited.add(friend)) {
                    queue.add(friend);
                    if (depth == 1) suggestions.add(friend);
                }
            }
        }
    }
    return suggestions;
}
```

Time complexity: O(V + E) using a queue

EVERY APP FEATURE:
UNDO AND REDO

STACK (LIFO)

Every action you take gets pushed onto a stack. Undo pops the last one off, redo pushes it right back.

JAVA

```
Deque undoStack = new ArrayDeque<>();
Deque redoStack = new ArrayDeque<>();

void doAction(String action) {
    undoStack.push(action);
    redoStack.clear();
}

void undo() {
    if (!undoStack.isEmpty())
        redoStack.push(undoStack.pop());
}

void redo() {
    if (!redoStack.isEmpty())
        undoStack.push(redoStack.pop());
}
```

Time complexity: O(1) for push and pop

GOOGLE FEATURE:

SEARCH AUTOCOMPLETE

TRIE

Every keystroke walks one level deeper into a Trie built from millions of past searches, instantly narrowing down matches.

JAVA

```
class TrieNode {
    Map children = new HashMap<>();
    boolean isEndOfWord;
}

void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        node = node.children
            .computeIfAbsent(c, k -> new TrieNode());
    }
    node.isEndOfWord = true;
}
```

Time complexity: O(L), where L is the word length

BROWSER FEATURE:
BACK AND FORWARD

TWO STACKS

Every page you visit gets pushed onto a back stack. Hitting back pops it off and pushes it onto a forward stack.

JAVA

```
Deque backStack = new ArrayDeque<>();
Deque forwardStack = new ArrayDeque<>();

void visit(String url) {
    backStack.push(url);
    forwardStack.clear();
}

String goBack() {
    forwardStack.push(backStack.pop());
    return backStack.peek();
}

String goForward() {
    String url = forwardStack.pop();
    backStack.push(url);
    return url;
}
```

Time complexity: O(1) per navigation

GOOGLE MAPS FEATURE:
SHORTEST ROUTE

GRAPH + DIJKSTRA

Roads become graph edges with travel time as weight. Dijkstra's algorithm finds the shortest path using a priority queue.

JAVA

```
PriorityQueue pq =
    new PriorityQueue<>((a, b) -> a[1] - b[1]);
pq.add(new int[]{source, 0});
dist[source] = 0;

while (!pq.isEmpty()) {
    int[] curr = pq.poll();
    for (int[] edge : graph.get(curr[0])) {
        int newDist = dist[curr[0]] + edge[1];
        if (newDist < dist[edge[0]]) {
            dist[edge[0]] = newDist;
            pq.add(new int[]{edge[0], newDist});
        }
    }
}
```

Time complexity: O(E log V)

SPOTIFY FEATURE:

PLAY NEXT QUEUE

QUEUE (FIFO)

Songs you add to the queue play in the exact order you added them. First one in is always the first one out.

JAVA

```
Queue playQueue = new LinkedList<>();

void addToQueue(String song) {
    playQueue.add(song);
}

String playNext() {
    return playQueue.poll();
}

String peekUpNext() {
    return playQueue.peek();
}
```

Time complexity: O(1) for add and poll

UBER FEATURE:

NEAREST DRIVER MATCH

PRIORITY QUEUE

Nearby drivers are indexed using a spatial structure, then ranked in a priority queue by distance to find your closest match instantly.

JAVA

```
PriorityQueue nearest = new PriorityQueue<>(
    (a, b) -> Double.compare(
        distance(a, rider), distance(b, rider)));

nearest.addAll(availableDrivers);
Driver closest = nearest.poll();
```

Time complexity: O(log N) per lookup

PHONE FEATURE:
RECENT APPS CACHE
HASHMAP + LINKED LIST

LinkedHashMap in access order mode behaves like a real LRU cache, automatically evicting the least recently used item.

JAVA

```
class LRUCache extends LinkedHashMap {
    int capacity;

    LRUCache(int capacity) {
        super(capacity, 0.75f, true);
        this.capacity = capacity;
    }

    protected boolean removeEldestEntry(
            Map.Entry eldest) {
        return size() > capacity;
    }
}
```

Time complexity: O(1) for get and put

KEYBOARD FEATURE:
AUTOCORRECT
TRIE + DYNAMIC PROGRAMMING

Your keyboard calculates edit distance between what you typed and real words in its Trie to guess the closest match.

JAVA

```
int editDistance(String a, String b) {
    int[][] dp = new int[a.length()+1][b.length()+1];
    for (int i = 0; i <= a.length(); i++) dp[i][0] = i;
    for (int j = 0; j <= b.length(); j++) dp[0][j] = j;

    for (int i = 1; i <= a.length(); i++) {
        for (int j = 1; j <= b.length(); j++) {
            if (a.charAt(i-1) == b.charAt(j-1))
                dp[i][j] = dp[i-1][j-1];
            else
                dp[i][j] = 1 + Math.min(dp[i-1][j-1],
                    Math.min(dp[i-1][j], dp[i][j-1]));
        }
    }
    return dp[a.length()][b.length()];
}
```

Time complexity: O(m x n), word lengths

FILE EXPLORER FEATURE:
FOLDER STRUCTURE

TREE

Every folder and subfolder you see is a tree, and File Explorer simply runs a depth first traversal to render it on screen.

JAVA

```
class Folder {
    String name;
    List subFolders = new ArrayList<>();

    void addSubFolder(Folder folder) {
        subFolders.add(folder);
    }
}

void printStructure(Folder folder, int depth) {
    System.out.println("  ".repeat(depth) + folder.name);
    for (Folder sub : folder.subFolders)
        printStructure(sub, depth + 1);
}
```

Time complexity: O(N) to traverse N folders